When to Think, When to Just Fuse: Adaptive LLM-Guided Operator Scheduling

MLSys 2026 Competition — Track B: Agent Reasoning

1. Introduction

Scheduling a DAG of MatMul and Pointwise ops onto an accelerator with limited scratchpad memory is a search problem over an exponential space. Our agent, **AccelerAgent**, was built around a single observation: for the vast majority of scheduling decisions, the roofline latency model gives an unambiguous answer. Only a small fraction involve genuine trade-offs that benefit from LLM reasoning. Spending 12 seconds on a Gemini call to confirm what arithmetic already made obvious is wasteful; spending those same 12 seconds where it matters can be decisive.

2. A Brief History of Each Optimization

2.1 Starting Point: Pure LLM Scheduling

Our initial agent (Run 1) was deliberately minimal: propose candidates, evaluate them, ask Gemini, apply, repeat. It worked but was slow. On the 103-op benchmark, 87 steps $\times$ 12 s per LLM call = nearly 18 minutes—well over the 9-minute wall limit. The first analytical win came from **snake traversal** (Run 2): row-major and column-major snake tile orders keep one matrix resident across a sweep, giving -26.7% on mlsys-2026-1 with zero LLM overhead.

2.2 Confidence-Based LLM Bypass

The breakthrough came from measuring how often Gemini actually *changed* the greedy decision: almost never on easy steps, occasionally on ambiguous ones. We formalized this as a confidence gate (Run 3, refined in Run 6):

$$\text{skip LLM} \iff \text{top_lat} \leq 0.70 \times \text{second_lat}$$

with additional overrides when only one candidate is feasible, the budget is exhausted, or the top two are tied. When the gate fires, the agent enters a fast-path batch loop at roughly 1 ms per step instead of 12 s. An unexpected consequence: with a 120-second test budget, the LLM stops being called after a few early decisions and the agent becomes nearly deterministic. We stopped chasing LLM tuning and focused on the deterministic fast path instead.

2.3 Tensor Retention and Recomputation

Runs 4–5 added two complementary memory optimizations. **Retention** (Run 4) keeps boundary output tensors in fast memory when a future step will consume them, scored by

$$(\text{reload_cost_saved} - \text{evict_cost_now}) \times \left(1 + \tfrac{1}{1+\text{topo_dist}}\right)$$

and retained greedily within a 60% capacity budget. The topo-distance factor prioritizes tensors consumed soon, avoiding a distant tensor squatting in scratchpad across many intermediate steps.

Recomputation (Run 5) targets diamond patterns (one producer, two or more consumers) by proposing FlashAttention-style alternatives: recompute the producer inside each consumer, fuse all consumers into one step, or create a mega-fusion that eliminates the intermediate tensor entirely.

2.4 Structural Local Search

After the greedy schedule converges, it is often stuck in a local optimum that re-tiling within steps cannot escape. Run 7 introduced a post-hoc structural optimizer with four move types:

1. **Merge**: fuse two adjacent steps if the merged subgraph is faster.
2. **Split**: break a high-working-set step at its largest boundary tensor.
3. **Swap**: reorder two adjacent independent steps to improve tensor residency.
4. **Flip**: drop retention from a step and re-optimize for larger tiles.

A subtle correctness issue lurked in Swap: the independence check must account for *orphan-promoted* tensors—ephemeral tensors (produced and consumed within a step) that are also consumed by the adjacent step. Swapping steps that share such a tensor makes it unavailable after the swap. A similar gap existed in Split, where the first half's boundary outputs omitted ephemerals needed by the second half. Both bugs were caught when residual-connection benchmarks triggered the promotion path.

2.5 DP for Linear Chains

Run 31 added a Phase 0.5 that runs before greedy: find maximal linear chains (each op has ≤ 1 remaining producer and ≤ 1 consumer) of length ≥ 3, and partition each chain optimally via DP in $O(n \cdot \texttt{max_len})$ time.

This produced an important false-start lesson. We originally ran DP on *all* discovered chains. But some chains feed **junction ops**—consumers that also depend on producers *outside* the chain. Pre-committing such a chain prevents the greedy from proposing a larger fusion spanning the junction. In a multi-head attention graph, each head's $Q \to QK \to$ softmax chain looks linear, yet the optimal schedule fuses it with the corresponding key matmul and attention-score op in one step, achieving 5$\times$ lower latency. Our junction-awareness check—"does the terminal op feed a consumer with remaining producers outside the chain?"—defers such chains to the greedy, which naturally discovers the larger fusion.

2.6 Global DP Repartition

Run 33 added a post-convergence step that re-runs DP over the entire committed op sequence, treating any contiguous block as a candidate fusion. The committed op *order* is sensible; only the step *boundaries* are reconsidered. This gave −22.7% on our mixed-topology synthetic benchmark (1,647,608 → 1,273,562) with zero regression on 22 others.

3. Agent Architecture

3.1 Pipeline Overview

Phase 0 (Strategy). Graph metrics, a probe evaluation, and a structured Gemini call select one of five preset strategies (`aggressive_fusion`, `balanced`, `conservative`, `memory_optimized`, `compute_optimized`), each a tuple of (`max_len`, `gran_top_k`, `retention_pct`). This is the highest-value LLM call: graph-level topology and memory pressure matter here in ways that local decisions cannot see.

Phase 0.5 (DP Chains). Discover junction-safe linear chains; commit their DP-optimal partitions before greedy begins.

Phase 1 (Multi-Restart Greedy). Three independent restarts with budget splits 45%/35%/20% explore different strategy configurations. A 1-step lookahead scores the top-5 distinct candidates before committing to ambiguous decisions. The best restart proceeds to Phase 2.

Phase 2 (Local Search). Multi-pass alternation of structural moves and per-step granularity/traversal re-optimization until convergence, followed by global DP repartition if budget remains.

3.2 Latency Model

Roofline per step: $\text{lat} = \sum_{\text{tiles}} \max(t_c, t_m)$, where $t_c = \text{base_cost}/(n_w \cdot n_h)$ and $t_m = (\text{slice_bytes_in} + \text{slice_bytes_out})/B$. Split-$K$ adds $k_{\text{steps}} = \lceil K/k \rceil$ passes over the reduction dimension, keeping the left-hand matrix fully resident. Tile slices are precomputed once per candidate and shared across all granularity evaluations.

3.3 Candidate Generation

Three sources feed the evaluation pipeline: (1) topological frontier expansion up to `max_len` ops; (2) a fanout/join recomputation solver that detects diamond patterns and proposes flash-style alternatives; (3) 1-step lookahead for ambiguous early decisions. Cache-aware ordering de-prioritizes candidates with cold tensor loads.

4. Results

Benchmark	Ops	Steps	Latency	Wall
example_problem	2	1	3,277	2.4 s
mlsys-2026-1	5	1	162,568	1.5 s
mlsys-2026-5	19	3	250,333	11.9 s
mlsys-2026-9	32	1	3,969,914	35.7 s
mlsys-2026-13	63	18	676,659	32.9 s
mlsys-2026-17	103	87	991,193	24.2 s

Table 1: Results on all 6 released benchmarks (all pass OOM validation).

All benchmarks complete well within the 9-minute limit. mlsys-2026-9 (32 ops) fuses to a single step at k=4—the largest granularity where the working set (229 KB) fits within 250 KB fast memory. We verified analytically that any multi-step partition incurs boundary tensor I/O exceeding the single-step cost. The agent also passes all 17 synthetic benchmarks spanning residual networks, transformer layers, wide-parallel fan-out, and mixed structures.

5. Discussion

The right role for an LLM. LLMs are most valuable at two specific moments: choosing a global strategy at the start and resolving genuinely ambiguous local decisions mid-schedule (candidates within 30% of each other). For everything else—granularity, traversal, retention, structural moves—the latency model gives a crisp answer in microseconds. An LLM agent that respects this boundary spends its reasoning budget wisely.

DP chains and junction awareness. The junction-feeding chain subtlety generalizes: any pre-commitment of a subgraph touching only *part* of a larger optimal fusion blocks that fusion from being discovered. The junction check is a precise structural predicate for detecting this without false positives—it fires on attention patterns but not on sequential pipelines, exactly where it should.

Structural local search as the real workhorse. In hindsight, the biggest performance gains came not from the LLM but from the deterministic post-hoc optimizer. Merge, split, swap, and flip moves escape local optima that neither granularity tuning nor traversal selection can reach, by exploring a structured neighborhood around the initial schedule.

6. Conclusion

AccelerAgent demonstrates that effective LLM-guided scheduling requires knowing when *not* to call the LLM. The combination of a tight analytical latency model, junction-aware DP chain pre-partitioning, multi-restart greedy with lookahead, structural local search, and global DP repartition produces competitive schedules efficiently and reliably across diverse graph topologies.